

ANA BÖLÜMLER (16 Adet)

- 1) **SQL Nedir? Temel Kavramlar** (Komut türleri, veritabanı mantığı, tablo yapısı, temel terimler)
- 2) **SELECT, WHERE, ORDER BY, LIKE**
- 3) **Aggregate Fonksiyonlar** (COUNT, SUM, AVG, MIN, MAX)
- 4) **JOIN'ler** (INNER, LEFT, RIGHT, FULL, CROSS, SELF)
- 5) **Primary Key, Foreign Key, İlişkiler** (1-1, 1-N, N-N)
- 6) **Subquery** (IN, EXISTS, Correlated Subquery)
- 7) **Set Operatörleri** (UNION, UNION ALL, INTERSECT, EXCEPT)
- 8) **Normalizasyon** (1NF, 2NF, 3NF) + **Denormalization**
- 9) **Index'ler** (Clustered, Nonclustered, Covering, Seek vs Scan)
- 10) **Query Execution Plan & Performans Tuning** (index kullanımı, cost analysis, bottleneck tanıma)
- 11) **Transaction, Lock, Deadlock, Isolation Levels** (ACID, lock türleri, deadlock çözüm mantığı, isolation seviyeleri)
- 12) **Stored Procedure, Function, Trigger** (Farkları, kullanım senaryoları, performans etkileri)
- 13) **CTE (WITH) Kullanımı** (Recursive CTE dahil)
- 14) **Window Functions (ROW_NUMBER, RANK, LAG, LEAD)** (Mülakatlarda üst seviye soru olarak gelir)
- 15) **Temp Table vs Table Variable** (Hangisi daha hızlı? Hangisi hafızayı nasıl kullanır?)
- 16) **View'ler** (Indexed View dahil)

BÖLÜM 1 — SQL NEDİR? (En Temel Seviye)

SQL nedir?

SQL (Structured Query Language), veri tabanlarıyla konuşmamızı sağlayan dildir.

Bir restorana garson üzerinden sipariş verdiğini düşün:

Sen = kullanıcı

Garson = SQL

Mutfak = veritabanı

Sen “Hamburger istiyorum” diyorsun → garson mutfağa söylüyor → mutfak hazırlayıp geri gönderiyor.

SQL de bunun aynısını veritabanında yapar.

SQL ne işe yarar?

SQL ile veritabanına:

- Veri ekleyebilirsin
- Veri silebilirsin
- Veri güncelleyebilirsin
- Veri sorgulayabilirsin
- Tablo oluşturabilirsin
- Tablo yapısını değiştirebilirsin

Bunların hepsi SQL komutlarıyla yapılır.

Veritabanı nedir?

Veritabanı, verilerin düzenli şekilde saklandığı dijital bir depodur. Excel gibi değil, çok daha güçlü yapılardır.

Tablo nedir?

Veritabanındaki veriler tablolar halinde saklanır. Excel’deki sayfalar gibi düşünebilirsin.

Bir çalışan tablosu örneği:

Id	Ad	Soyad	Maas
1	Ali	Yılmaz	30000
2	Ayşe	Demir	35000

Temel SQL Komut Türleri

SQL komutlarını 4 ana grupta toplarız:

1) DQL — Data Query Language

Veri sorgulama

SELECT

2) DML — Data Manipulation Language

Veriyi değiştirme

INSERT, UPDATE, DELETE

3) DDL — Data Definition Language

Tablo yapısını yönetme

CREATE, ALTER, DROP

4) DCL / TCL — Yetki ve Transaction kontrol

GRANT, REVOKE, COMMIT, ROLLBACK

(Mülakatta bunlar çok sorulur!)

BÖLÜM 2 — SELECT Komutu (Sorgulama Temelleri)

SELECT = Veritabanından veri okumak için kullanılır.

En basit hali:

```
SELECT * FROM Employees;
```

Anlamı:

👉 Employees tablosundaki tüm verileri getir.

1) Belirli Kolonları Seçme

Her şeyi getirmek yerine sadece istediğin kolonları çekebilirsin:

```
SELECT FirstName, LastName FROM Employees;
```

2) WHERE ile Filtreleme

Filtreleme yapar:

```
SELECT * FROM Employees  
WHERE Salary > 30000;
```

Anlamı:

👉 Maaşı 30000'den yüksek olan çalışanları getir.

Küçük operatör hatırlatması:

Operatör	Açıklama
=	eşit
>	büyük
<	küçük
>=	büyük eşit
<=	küçük eşit
!= veya <>	eşit değil

3) AND / OR ile Koşulları Birleştirme

```
SELECT * FROM Employees  
WHERE Salary > 30000 AND Department = 'IT';
```

- **AND** = her iki şart da doğru olmalı
 - **OR** = şartlardan biri doğru olsa yeter
-

4) LIKE ile Arama (Mülakatta kesin çıkar)

Metinsel aramalar:

```
SELECT * FROM Employees  
WHERE FirstName LIKE 'A%';
```

Anlamı:

👉 Adı A ile başlayanlar.

Wildcard karakterler:

- % = sıfır veya daha fazla karakter
- _ = tek karakter

Örnek:

```
WHERE LastName LIKE '%mir'    -- sonu 'mir' ile bitenler  
WHERE FirstName LIKE '_li'    -- 3 harfli, ortası 'l' olanlar
```

5) ORDER BY (Sıralama)

```
SELECT * FROM Employees  
ORDER BY Salary DESC;    -- büyükten küçüğe
```

Default sıralama: ASC (küçükten büyüğe)

6) LIMIT / TOP (İlk X Kaydı Getirme)

SQL Server:

```
SELECT TOP 5 * FROM Employees;
```

MySQL / PostgreSQL:

```
SELECT * FROM Employees LIMIT 5;
```

7) DISTINCT — Tekil Kayıtlar

Aynı değerleri tekrar ettirmeden getirir.

```
SELECT DISTINCT Department FROM Employees;
```

✅ MÜLAKAT SORULARI (SELECT Bölümü)

1) SELECT * kullanmak doğru mudur?

Cevap:

- “*” kullanmak performansı düşürebilir.
 - Gereksiz veri taşınır.
 - Index kullanımı etkilenebilir.
 - Mülakatta doğru cevap:
👉 “Üretim ortamında SELECT * kullanılmaz, sadece gerekli kolonlar seçilir.”
-

2) WHERE ile HAVING farkı nedir? (çok sorulur)

WHERE

Gruplamadan önce filtreler
Normal kolonlarda çalışır

HAVING

Gruplamadan sonra filtreler
Genellikle aggregate fonksiyonlarla kullanılır

Örnek:

```
SELECT Department, COUNT(Id)
FROM Employees
GROUP BY Department
HAVING COUNT(Id) > 5;
```

3) LIKE ile % ve _ farkı nedir?

Cevap:

- % = 0 veya daha fazla karakter
 - _ = tam 1 karakter
-

4) = ve LIKE arasındaki fark?

Cevap:

- = kesin eşleşme
 - LIKE benzer eşleşme (pattern matching)
-

5) ORDER BY ASC ve DESC ne işe yarar?

- ASC → küçükten büyüğe
 - DESC → büyükten küçüğe
- Default: ASC
-

6) DISTINCT ne işe yarar?

Aynı değerleri tekrar göstermeden listeleme yapar.

7) Filtrelemede AND ve OR nasıl çalışır?

- AND → tüm şartlar sağlanmalı
- OR → en az biri sağlanmalı

BÖLÜM 3 — Aggregate (Toplulaştırma) Fonksiyonlar

Aggregate fonksiyonlar, **bir grup veriyi özetlemek** için kullanılır. Yani satır satır değil, toplu hesaplama yaparlar.

Temel fonksiyonlar:

- **COUNT** → adet
- **SUM** → toplam
- **AVG** → ortalama
- **MIN** → minimum
- **MAX** → maksimum

Hepsi SELECT ile kullanılır.

1) COUNT() — Adet Bulma

```
SELECT COUNT(*) FROM Employees;
```

👉 Kaç çalışan var?

Belirli bir kolon üzerinden de sayabilirsin:

```
SELECT COUNT(Department) FROM Employees;
```

Fark:

- `COUNT(*)` → tüm satırları sayar
 - `COUNT(column)` → NULL olmayanları sayar
-

2) SUM() — Toplam Alma

```
SELECT SUM(Salary) FROM Employees;
```

👉 Tüm çalışanların toplam maaşı.

3) AVG() — Ortalama

```
SELECT AVG(Salary) FROM Employees;
```

4) MIN() ve MAX() — En küçük / En büyük

```
SELECT MIN(Salary), MAX(Salary)
FROM Employees;
```

5) WHERE ile birlikte kullanım

```
SELECT AVG(Salary)
FROM Employees
WHERE Department = 'IT';
```

👉 Sadece IT departmanının ortalama maaşı.

6) GROUP BY (Çok çok önemli!)

Aggregate fonksiyonları **gruplamak için** kullanırız.

Örnek:

```
SELECT Department, COUNT(*)
FROM Employees
GROUP BY Department;
```

Bu sorgu:

Department Count

IT	5
HR	2
Finance	3

gibi sonuç döndürür.

7) HAVING — Gruplanmış veriyi filtreler

HAVING, GROUP BY'dan sonra **aggregate sonuçları filtrelemek için** kullanılır.

Örnek:

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 3;
```


👉 3'ten fazla çalışanı olan departmanları getir.

WHERE ile HAVING farkı mülakatlarda çok sorulur ve önceki bölümde anlattık:

- WHERE → gruplamadan önce
 - HAVING → gruplamadan sonra
-

8) GROUP BY ve SELECT ilişkisinde mülakat sorusu

“SELECT kısmında yazılan her kolon GROUP BY’da olmalı mı?”

Cevap:

- Evet, SQL Server ve PostgreSQL’de **aggregate olmayan tüm kolonlar GROUP BY’da olmak zorunda.**
 - MySQL’de ise “ONLY_FULL_GROUP_BY” kapalıysa zorunlu değildir ama bu **kötü pratiktir.**
-

🎯 MÜLAKATTA AGGREGATE FONKSİYONLARI ÜZERİNE SIK SORULAN SORULAR

1) COUNT(*) ve COUNT(column) farkı nedir?

Cevap:

- COUNT (*) tüm satırları sayar
 - COUNT(column) sadece NULL olmayan kolonları sayar
-

2) GROUP BY ne işe yarar?

Cevap:

Verileri belirli bir kolona göre gruplar.

Örneğin departman bazlı çalışan sayısı gibi.

3) HAVING neden vardır?

Cevap:

Gruplanmış sonuçlara filtre eklemek için.

4) WHERE ve HAVING farkı?

Bu soruyu ezberle:

- WHERE → satırları filtreler (gruplamadan önce)
 - HAVING → grupları filtreler (gruplamadan sonra)
-

5) MIN/MAX ile WHERE kullanılabilir mi?

Doğrudan hayır, genelde subquery gerekir.

Örnek:

```
SELECT * FROM Employees  
WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

6) SUM() kullanırken NULL değerler ne olur?

NULL değerler toplama dahil edilmez.

7) AVG() hesaplamasında NULL nasıl davranır?

NULL değerleri yok sayılır.

BÖLÜM 4 — JOIN NEDİR?

JOIN, iki veya daha fazla tabloyu birleştirip tek bir sonuç üretmek için kullanılır.

Her tabloda ortak bir kolon olur (genelde Id / Foreign Key).

Örnek senaryo:

Employees

Id	Name	DeptId
1	Ali	1
2	Ayşe	2
3	Mehmet	1

Departments

Id	DepartmentName
1	IT
2	HR

1) INNER JOIN (Kesişim – en çok kullanılan)

```
SELECT e.Name, d.DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DeptId = d.Id;
```

👉 Her iki tabloda eşleşen kayıtları getirir.

Görsel mantık:

Employees $\cap$ Departments (sadece ortak olanlar)

2) LEFT JOIN (solun tamamı + sağın eşleşenleri)

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
    ON e.DeptId = d.Id;
```

👉 Sol tabloda (Employees) ne varsa hepsi gelir.
SAĞ tablodaki eşleşmeyenler NULL olur.

Örnek:

Bir çalışanın DeptId'si yanlış veya boş olabilir → yine gelir.

3) RIGHT JOIN (sağın tamamı + solun eşleşenleri)

```
SELECT e.Name, d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
    ON e.DeptId = d.Id;
```

👉 Sağ tablodaki (Departments) her şey gelir.
Sol tablodan eşleşmeyenler NULL olur.

(Mülakatta söylenir: RIGHT JOIN çok nadir kullanılır.)

4) FULL JOIN (her iki tablonun tümü)

```
SELECT e.Name, d.DepartmentName
FROM Employees e
FULL JOIN Departments d
    ON e.DeptId = d.Id;
```

👉 Tüm kayıtlar gelir
Eşleşmeyenler → NULL.

Mantığı:

Employees $\cup$ Departments (her şey)

🟡 5) CROSS JOIN (kombinasyon, çarpazlama)

```
SELECT * FROM A CROSS JOIN B;
```

👉 A'daki her satır, B'deki her satır ile eşleşir.

Genelde raporlama için kullanılır, dikkat: kayıt sayısı patlar.

🟢 6) SELF JOIN (tablonun kendisiyle join)

Örnek: bir çalışanın yöneticisi yine Employees tablosunda.

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m
    ON e.ManagerId = m.Id;
```

🎯 JOIN + WHERE + GROUP BY örneği (Mülakat seviyesi)

Departman bazlı ortalama maaş:

```
SELECT d.DepartmentName, AVG(e.Salary)
FROM Employees e
INNER JOIN Departments d
    ON e.DeptId = d.Id
GROUP BY d.DepartmentName;
```

🔥 JOIN ÜZERİNE MÜLAKATTA GELEN SORULAR + CEVAPLAR

1) INNER JOIN ile LEFT JOIN farkı nedir? (kesin soru)

INNER JOIN: sadece eşleşen kayıtları getirir.

LEFT JOIN: sol tablodaki tüm kayıtlar gelir, sağ eşleşmeyenler NULL olur.

2) RIGHT JOIN neden çok az kullanılır?

Cevap:

- Veri modellerinde genelde “sol taraf ana tablo” gibi düşünülür,
- LEFT JOIN daha okunaklıdır.

- RIGHT JOIN gerekli çoğu durumda LEFT JOIN ters çevrilerek çözülebilir.

3) FULL JOIN ne zaman kullanılır?

Cevap:

- Veri uyumsuzluğu olduğunda
 - Her iki taraftaki eşleşmeyen kayıtları da görmek için
Ör: Sistem karşılaştırma, kalite kontrol raporları.
-

4) CROSS JOIN nedir? Tehlikesi nedir?

Cevap:

- Tüm satırlar birbirleriyle eşleşir.
 - $1000 \text{ kayıt} \times 1000 \text{ kayıt} = 1.000.000$ sonuç döner $\rightarrow$ performans patlar.
-

5) SELF JOIN nedir? Ne zaman kullanılır?

Cevap:

Aynı tablodan hiyerarşik ilişki çıkarılması gerektiğinde.

Ör: çalışan-yönetici ilişkisi.

6) JOIN mi hızlıdır yoksa SUBQUERY mi? (çok önemli!)

Genelde:

- **JOIN daha hızlıdır**
- SUBQUERY bazen yavaş olabilir (özellikle correlated subquery)

Mülakat cevabı:

👉 “JOIN ile tabloyu genişletmek veritabanı açısından daha optimize bir işlemdir.”

BÖLÜM 5 — Primary Key, Foreign Key ve Tablolar Arası İlişkiler

Veritabanında tablolar **tek başına değil**, ilişkili şekilde çalışır.
Bu ilişkileri kurmak için **Primary Key** ve **Foreign Key** kullanırız.

1) PRIMARY KEY (PK)

Primary Key nedir?

Bir tablodaki **her satırı benzersiz şekilde tanımlayan kolondur.**

Özellikleri:

- Tekil olmalı (unique)
- NULL olamaz
- Genelde Id kolonudur
- Bir tabloda sadece 1 tane PK olabilir (ama PK birden fazla kolon da olabilir → Composite Key)

Örnek:

```
CREATE TABLE Employees (  
    Id INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Salary INT  
);
```

2) FOREIGN KEY (FK)

Foreign Key nedir?

Bir tablodaki kolonun **başka bir tablodaki Primary Key'i işaret etmesidir.**

Yani iki tabloyu bağlayan köprü.

Örnek:

```
CREATE TABLE Departments (  
    Id INT PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

```
CREATE TABLE Employees (  
    Id INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DeptId INT,  
    FOREIGN KEY (DeptId) REFERENCES Departments (Id)  
);
```

Bu demektir ki:

👉 Employees.DeptId → Departments.Id kolonuna bağlıdır.

3) İlişki Türleri (One-to-One, One-to-Many, Many-to-Many)

A) One-to-One (1'e 1 ilişki)

Bir kişi → Bir kimlik kartı

Ör:

Users → UserDetails

Users.Id = UserDetails.UserId

Mülakatta sorulur:

👉 “1’e 1 ilişkilerde genelde ikinci tabloya PK = FK yapılır.”

B) One-to-Many (1'e çok ilişki)

EN ÇOK KULLANILAN.

Bir departman → birçok çalışan

Departments (1) → Employees (many)

Departments.Id → Employees.DeptId

Bu ilişki neredeyse tüm projelerde vardır.

C) Many-to-Many (Çoka çok ilişki)

Bir öğrenci birçok kursa katılabilir,

Bir kursa birçok öğrenci katılabilir.

Bu durumda **ara tablo** kurulur:

- Students
- Courses
- StudentCourses (JOIN TABLE)

Örnek:

```
CREATE TABLE StudentCourses (  
    StudentId INT,  
    CourseId INT,  
    PRIMARY KEY (StudentId, CourseId),  
    FOREIGN KEY (StudentId) REFERENCES Students (Id),  
    FOREIGN KEY (CourseId) REFERENCES Courses (Id)  
);
```

4) Cascade Kuralları (ON DELETE CASCADE / ON UPDATE CASCADE)

Foreign key verirken ekstra kurallar ekleyebilirsiniz.

ON DELETE CASCADE

Parent silinirse child kayıtları da otomatik silinir.

```
FOREIGN KEY (DeptId) REFERENCES Departments (Id)  
ON DELETE CASCADE
```

ON UPDATE CASCADE

Parent'ın Id'si güncellenirse child taraf da güncellenir.

Gerçek hayatta ON UPDATE çok az kullanılır.

5) PK - FK tasarımında en sık yapılan hatalar

NULL FK bırakmak

Bazen gerekli olur ama çoğu zaman tasarım hatasıdır.

Cascade'i yanlış kullanmak

Yanlışlıkla tüm alt kayıtlar silinebilir → facia.

✗ Many-to-Many ilişkide ara tablo oluşturmamak

Bu büyük bir mimari hatadır.

✗ PK olarak anlamlı kolon kullanmak

TC Kimlik, e-mail vb.

Doğrusu:

👉 Her zaman anlamsız, otomatik artan Id kullanmak.

🎯 MÜLAKATTA SIK SORULAN SORULAR + CEVAPLAR

1) Primary Key ile Unique Key farkı nedir?

Primary Key:

- Sadece 1 tane olabilir
- NULL olamaz

Unique Key:

- Bir tabloda birden fazla olabilir
 - NULL olabilir (SQL Server hariç çoğu DB'de)
-

2) Foreign Key ne işe yarar?

Cevap:

- Veri bütünlüğünü sağlar
 - Yanlış DeptId girilmesini engeller
 - İlişkileri garanti altına alır
-

3) One-to-Many ilişkii nasıl kurarsın?

Cevap:

- “One” taraf → Primary Key
 - “Many” taraf → Foreign Key
- Ör:
Departments.Id → Employees.DeptId
-

4) Many-to-Many nasıl kurulur?

Cevap:

Ara tabloyla:

StudentCourses(StudentId, CourseId)

5) Cascade nedir? ON DELETE CASCADE ne işe yarar?

Cevap:

Parent kayıt silindiğinde child kayıtların otomatik silinmesini sağlar.

6) Composite Key nedir?

Birden fazla kolonun bir araya gelerek Primary Key oluşturmasıdır.

Ör: StudentId + CourseId.

7) PK olarak e-mail, TC, telefon kullanılır mı?

Cevap:

Hayır, **kullanılmamalıdır**.

Çünkü değişebilir veya güvenlik açıklarına yol açar.

En doğru PK → anlamsız numeric Id.

BÖLÜM 6 — SUBQUERY (İç İçe Sorgular)

Subquery, bir sorgunun **içinde başka bir sorgu çalıştırmaktır**.

SQL'deki yapısı:

```
SELECT ...  
FROM ...  
WHERE ... (SELECT ...)
```

1) Subquery Nerelerde Kullanılır?

- WHERE içinde
 - FROM içinde (Derived Table)
 - SELECT içinde (Scalar Subquery)
-

2) WHERE İçinde Subquery

En yaygın kullanımı.

Örnek:

En yüksek maaşı alan çalışanı getir:

```
SELECT *  
FROM Employees  
WHERE Salary = (  
    SELECT MAX(Salary) FROM Employees  
);
```

3) IN ile Subquery

```
SELECT *  
FROM Employees  
WHERE DeptId IN (  
    SELECT Id FROM Departments WHERE Location = 'ISTANBUL'  
);
```

Mantığı:

👉 İstanbul'daki departmanlara bağlı çalışanları getir.

🔵 4) EXISTS Subquery (Performans İçin Çok Önemli!)

EXISTS, **kayıt var mı yok mu** diye kontrol eder.

```
SELECT *
FROM Employees e
WHERE EXISTS (
    SELECT 1
    FROM Salaries s
    WHERE s.EmployeeId = e.Id
);
```

Vardır/yoktur kontrolünde en hızlı yöntemdir.
JOIN yerine EXISTS kullanılır.

🔵 5) FROM İçinde Subquery (Derived Table)

Subquery sonucunu tablo gibi kullanırsın.

```
SELECT d.DepartmentName, t.AverageSalary
FROM (
    SELECT DeptId, AVG(Salary) AS AverageSalary
    FROM Employees
    GROUP BY DeptId
) t
INNER JOIN Departments d ON d.Id = t.DeptId;
```

🔵 6) SELECT İçinde Subquery (Scalar Subquery)

Tek bir değer döner.

```
SELECT
    Name,
    (SELECT AVG(Salary) FROM Employees) AS OverallAvgSalary
FROM Employees;
```

🔥 7) Correlated Subquery (Bağımlı Subquery)****— Mülakat Seviyesi

İçteki query dıştaki query'nin satırına bağlıdır.

Örnek:

Bir çalışanın maaşı, kendi departman ortalamasının üzerinde mi?

```
SELECT e.Name, e.Salary
FROM Employees e
```

```
WHERE Salary > (  
    SELECT AVG(Salary)  
    FROM Employees  
    WHERE DeptId = e.DeptId  
) ;
```

Mantık:

👉 Her satır için subquery tekrar çalışır.

Performans olarak pahalıdır, join ile çözmek daha iyidir.

🎯 Mülakatlık Subquery + JOIN Karşılaştırması

Soru: “Subquery mi daha hızlıdır, JOIN mi?”

Cevap:

- JOIN genellikle daha hızlıdır
- Correlated Subquery yavaştır
- EXISTS genelde IN'den daha performanslıdır
- Subquery okunabilirliği artırabilir

Bu cevap mülakatlarda çok iş görür.

🎯 SUBQUERY ÜZERİNE MÜLAKAT SORULARI + CEVAPLAR

1) Subquery nedir?

Başka bir SQL sorgusunun içinde kullanılan sorgudur.

2) Correlated Subquery nedir?

Dıştaki sorgunun her satırı için tekrar çalışan subquery'dir.

3) IN ile EXISTS farkı? (sık sorulur)

- IN → alt sorgunun sonucu liste döndürür

- EXISTS → kayıt var mı yok mu kontrol eder
Performans olarak EXISTS üstün.

Örnek:

EXISTS daha hızlıdır çünkü ilk doğru sonucu bulunca durur.

4) JOIN mı kullanırsın SUBQUERY mi?

Cevap:

- “Performans için genellikle JOIN tercih ederim.”
 - “Okunabilirlik için SUBQUERY de kullanılabilir.”
-

5) Scalar Subquery nedir?

SELECT içinde tek bir değer döndüren subquery.

6) FROM içinde subquery kullanmak ne işe yarar?

Geçici tablo oluşturur (Derived Table).

BÖLÜM 7 — SET OPERATÖRLERİ

Set operatörleri, **iki ayrı SELECT sonucunu birleştirmek** için kullanılır.
Ancak dikkat:

👉 **Her iki SELECT’deki kolon sayısı ve kolon tipleri aynı olmalıdır.**
Bu mülakatlarda çok sorulan bir kural.

1) UNION (Tekilleştirerek Birleştirir)

```
SELECT Name FROM Employees_2023
UNION
SELECT Name FROM Employees_2024;
```

Özellikleri:

- Aynı isim birden fazla geliyorsa → **tekilleştirir**.
 - Performans olarak UNION ALL'dan **daha yavaştır**.
 - Sonuçlar varsayılan olarak **sıralanmış** gelir.
-

2) UNION ALL (Tekilleştirmez – daha hızlıdır)

```
SELECT Name FROM Employees_2023
UNION ALL
SELECT Name FROM Employees_2024;
```

Özellikleri:

- Tekilleştirme yok.
 - En hızlı set operatörüdür.
 - Özellikle log, rapor, birleşim işlerinde tercih edilir.
-

3) INTERSECT (Kesişim – ortak kayıtlar)

```
SELECT Name FROM Employees_2023
INTERSECT
SELECT Name FROM Employees_2024;
```

Özellikleri:

👉 Her iki sorguda da olan ortak değerleri getirir.

Örnek anlamı:

Hem 2023'te hem 2024'te çalışanlar.

4) EXCEPT (Fark kümesi)

```
SELECT Name FROM Employees_2023
EXCEPT
SELECT Name FROM Employees_2024;
```


Özellikleri:

☞ Sadece ilk sorguda olup ikinci sorguda olmayan kayıtları getirir.

Örnek anlamı:

2023'te çalışıp 2024'te çalışmayanlar.

🧠 Set Operatörleri Kullanırken Önemli Kurallar (Mülakatlık)

Kural	Açıklama
Kolon sayısı aynı olmalı	2 SELECT aynı sayıda kolon döndürmeli
Kolon tipleri uyumlu olmalı	INT → INT, VARCHAR → VARCHAR gibi
Sonuç kolonlarının isimleri	İlk SELECT'in kolon isimleri kullanılır
ORDER BY nereye yazılır?	EN SON'a yazılır

Örnek:

```
SELECT Name FROM Employees_2023
UNION ALL
SELECT Name FROM Employees_2024
ORDER BY Name;
```

🎯 MÜLAKATTA SIK SORULAN SORULAR + CEVAPLAR

1) UNION mı hızlıdır UNION ALL mı? (kesin soru)

Cevap:

UNION ALL daha hızlıdır çünkü:

- Tekilleştirme yapmaz
- Sıralama yapmaz

2) UNION ile UNION ALL arasındaki fark?

- UNION → duplicate kayıtları temizler
- UNION ALL → olduğu gibi getirir

3) INTERSECT ne işe yarar?

Her iki sorguda da ortak olan kayıtları getirir.

4) EXCEPT ne işe yarar?

İlk sorguda olup ikinci sorguda olmayan kayıtları getirir.

5) Set operatörlerinde kolon sayısı farklı olursa ne olur?

Hata verir.

Mülakat cevabı:

👉 “Set operatörlerinde her iki SELECT aynı kolon sayısında ve uyumlu tiplerde olmalıdır.”

6) ORDER BY neden sadece en sonda yazılır?

Çünkü set operatörleri tüm result set'i birleşmiş olarak sıralar.
Aksi halde hata alırsın.

BÖLÜM 8 — NORMALİZASYON (1NF, 2NF, 3NF)

Normalizasyon, veritabanını **daha düzenli, daha hızlı ve daha tutarlı** hale getirmek için kullanılan kurallar bütünüdür.

Amaçlar:

- Gereksiz tekrarları (redundancy) azaltmak
- Güncelleme hatalarını (anomalies) engellemek
- Tutarlı veri modeli oluşturmak

Normalizasyon seviyeleri $1NF \rightarrow 2NF \rightarrow 3NF$ şeklinde ilerler.

Bunlar mülakatta mutlaka çıkar.

1) 1NF — First Normal Form (Birinci Normal Form)

Kurallar:

1. Her kolon atomik (bölünemez) olmalı
2. Tekrar eden kolon grupları olmamalı
3. Her satır benzersiz olmalı (PK olmalı)

Kötü Tasarım:

Id | Name | Phone1 | Phone2 | Phone3

Neden yanlış?

- Tekrar eden kolonlar var.

Doğru Tasarım:

Phones diye ayrı tablo olmalı:

Employees

Id | Name

Phones

Id | EmployeeId | Phone

2) 2NF — Second Normal Form (İkinci Normal Form)

Şartlar:

- 1NF sağlanmış olmalı
- **Partial Dependency olmamalı**
(Yani PK birden fazla kolondan oluşuyorsa → diğer kolonlar PK'nin *tamamına* bağlı olmalı.)

Genelde **Composite Key** kullanılan tablolarda ortaya çıkar.

Kötü Tasarım:

StudentId + CourseId (PK)
CourseName

CourseName $\rightarrow$ CourseId'ye bağılı
Ama StudentId ile ilgisi yok $\rightarrow$ Partial Dependency var.

✓ Doğru Tasarım:

Course'ları ayrı tabloya almak:

Courses

CourseId | CourseName

3) 3NF — Third Normal Form (Üçüncü Normal Form)

Şartlar:

- 2NF sağlanmalı
- **Transitive Dependency olmamalı**
(Kolon, PK'ye değil başka bir kolona bağılı olmayacak.)

✗ Kötü Tasarım:

Employee(Id, Name, DepartmentId, DepartmentName)

DepartmentName $\rightarrow$ DepartmentId'ye bağılı
Ama Id'ye değil $\rightarrow$ Transitive Dependency var.

✓ Doğru Tasarım:

Departments

Id | DepartmentName

Employees

Id | Name | DepartmentId

🧠 Normalizasyonun Gerçek Hayattaki Faydaları

- Veri tekrarını azaltır
- Güncelleme problemlerini engeller
- Veri bütünlüğünü artırır
- Veritabanının büyüklüğünü küçültür
- JOIN performansı artar

⚠ NOT (Gerçek Hayat Perspektifi — Mülakat İçin Çok Değerli)**

Bazı durumlarda **denormalization** (bilerek normal kuralları bozma) yapılır:

- Raporlama (OLAP) sistemlerinde
- Çok fazla JOIN atılan senaryolarda
- Cache'lerde

Mülakatta şöyle dersin:

👉 “OLTP sistemlerinde normalizasyon idealdir, fakat raporlama/analitik sistemlerde denormalization performansı artırır.”

🎯 MÜLAKATTA SIK SORULAN SORULAR + CEVAPLAR

1) Normalizasyon nedir?

Veri tekrarını azaltmak ve veri bütünlüğünü artırmak için veritabanını kurallara göre düzenlemektir.

2) 1NF – 2NF – 3NF arasındaki fark nedir?

Cevap:

- 1NF → Kolonlar atomik olacak, tekrar eden kolon yok
 - 2NF → Composite Key kullanıyorsan partial dependency olmayacak
 - 3NF → Transitive dependency olmayacak
-

3) Transitive dependency nedir?

Bir kolon PK'ye değil başka bir kolona bağımlıysa.

4) Denormalization nedir? Ne zaman yapılır?

Normalizasyonu bilinçli olarak bozmak.

Raporlama ve performans gerektiren JOIN ağır tablolarda yapılır.

5) Composite Key nedir?

Birden fazla kolonun birleşerek Primary Key oluşturmasıdır.

6) Normalizasyon performansı artırır mı düşürür mü?

- OLTP sistemlerde artırır
 - Çok JOIN'li bazı sorgularda düşürebilir
Ama veri kalitesi açısından çok önemlidir.
-

BÖLÜM 9 — INDEX'LER

Index, veritabanında **arama hızını artırmak için** kullanılan veri yapısıdır.
Kitap arkasındaki **alfabetik fihrist** gibi düşünebilirsiniz.

- Index yoksa → SQL tüm kitabı baştan sona okur (**TABLE SCAN**)
 - Index varsa → direkt ilgili sayfaya gider (**INDEX SEEK**)
-

1) CLUSTERED INDEX (Kümeleştirilmiş Index)

Tablonun fiziksel sıralama şeklidir.

Yani satırlar diskte bu index'e göre fiziksel olarak sıralanır.

Özellikleri:

- Bir tabloda **sadece 1 tane** olabilir
- Genelde PK üzerinde olur
- Tablonun kendisidir → ekstra bir yapı değildir
- Arama en hızlı bu index ile yapılır

Örnek:

```
CREATE CLUSTERED INDEX IX_Employees_Id  
ON Employees (Id);
```

Mantık:

Employees tablosu Id değerine göre fiziksel olarak sıralanır.

 2) NONCLUSTERED INDEX

Tablodan ayrı bir yapıda tutulur.

Kitabın arkasındaki fihrist gibi çalışır.

Özellikleri:

- Bir tabloda **birden fazla** olabilir
- Sadece pointer/işaretçi taşır
- Arama hızını ciddi artırır
- Genelde sık sorgulanan kolonlara konur

Örnek:

```
CREATE NONCLUSTERED INDEX IX_Employees_Name  
ON Employees (Name) ;
```

 3) COVERING INDEX (En sevilen performans tekniği)

Bir sorgunun ihtiyaç duyduğu **tüm kolonlar index içinde** yer alıyorsa buna covering denir.

Örnek sorgu:

```
SELECT Name, Salary  
FROM Employees  
WHERE DepartmentId = 5;
```

Performansı artırmak için:

```
CREATE NONCLUSTERED INDEX IX_Employees_DepartmentId  
ON Employees (DepartmentId)  
INCLUDE (Name, Salary);
```

Artık tabloya gitmeden **tamamen index üzerinden** sonuç döner → çok hızlı.

4) INDEX SEEK vs INDEX SCAN vs TABLE SCAN

TABLE SCAN

Tablonun tüm satırları okunur → **en yavaş**.

INDEX SCAN

Index var ama veri çok geniş → index satır satır taranır.

INDEX SEEK

En hızlısı.

Index doğrudan ilgili değere “zıplar”.

Mülakat sorusu:

👉 “Index oluşturduktan sonra performans neden artar?”

Cevap: Çünkü TABLE SCAN → INDEX SEEK’e dönüşür.

5) Hangi Kolonlara Index Konulur? (Mülakatın en kritik sorusu)

Index koyulması gereken kolonlar:

- WHERE kullanılan kolonlar
- JOIN yapılan kolonlar
- GROUP BY yapılan kolonlar
- ORDER BY yapılan kolonlar
- Yüksek selectivity olan kolonlar (çok farklı değer içeren)

Index koyulmaması gereken kolonlar:

- Çok sık güncellenen kolonlar (örn: last_login_date)
- Çok düşük selectivity kolonlar (örn: Cinsiyet, AktifMi)
- Büyük text kolonlar (nvarchar(max), text)

6) Selectivity Nedir? (Mülakat düzeyi)

Bir kolondaki farklı değer oranıdır.

Örnek:

- Cinsiyet → Male/Female (düşük selectivity)
- TC Kimlik → her kişi farklı (yüksek selectivity)

Selectivity yüksek = Index verimli

7) Composite Index (Çok kolonlu index)

```
CREATE INDEX IX_Test  
ON TableA (Col1, Col2, Col3)
```

Kural:

SQL index'i **soldan sağa** okur.

Yani Col1 olmadan Col2 kullanılamaz.

MÜLAKATTA SIK SORULAN SORULAR + CEVAPLAR

1) Clustered index ile nonclustered index farkı?

Clustered:

- Tablonun fiziksel sıralaması
- 1 tane olur
- En hızlı aramayı sağlar

Nonclustered:

- Ayrı bir yapı
 - Birden fazla olabilir
-

2) Index seek ve index scan farkı?

- **Seek** → nokta atışı (direkt bulur)
 - **Scan** → satır satır tarama
-

3) Hangi kolonlara index konur?

WHERE, JOIN, ORDER BY, GROUP BY kullanılan kolonlara.

4) Too many indexes iyi mi?

Hayır.
INSERT/UPDATE/DELETE işlemlerini yavaşlatır.

5) Covering index nedir?

Sorgu için tüm kolonları içinde bulunduran index.

6) Selectivity nedir?

Kolondaki farklı değer oranı.
Selectivity yüksekse index çok verimli çalışır.

BÖLÜM 10 — QUERY EXECUTION PLAN & PERFORMANS TUNING

Execution Plan, SQL Server'ın bir sorguyu **NASIL çalıştırdığını** gösteren yol haritasıdır.

Bir nevi SQL Server'ın “trafik güzergâhı”.

1) Execution Plan Nedir?

SQL'in:

- hangi index'i kullandığını
- sıra olarak nasıl çalıştığını

- hangi adımın pahalı olduğunu
- performans darboğazlarının nerede olduğunu

gösteren plandır.

SQL Server Management Studio’da:

- ✓ **Actual Execution Plan**
- ✓ **Estimated Execution Plan**

ikisi de inceleme için kullanılır.

● 2) Execution Plan’de Bakılması Gereken En Önemli Şeyler

A) Index Seek varsa iyi

Genelde hızlıdır.

B) Index Scan veya Table Scan varsa problem olabilir

Tüm tabloyu tarar → yavaş.

C) Key Lookup (RID Lookup)

Nonclustered index kullanılırken tablodan ek kolon alınması gerekiyorsa olur.

Key Lookup çok sık olursa → performans düşer.

Çözüm: Covering Index.

D) Sort (Pahalı Operasyon)

ORDER BY olmayan bir tabloda sıralama yapmak maliyetlidir.

E) Hash Match, Merge Join, Nested Loop

Join operatorleri:

- **Nested Loop** → küçük tablolarda hızlı
- **Merge Join** → sırayla okur, orta düzey hızlı
- **Hash Match** → büyük tablolar için ama maliyetlidir

F) Missing Index Hints

Plan bazen şöyle yazar:

Missing Index (Impact 95%)

Bu gerçekten çok büyük bir ipucudur.

3) En Önemli Performans Problemleri ve Çözümler

1) WHERE kolonunda index yok

Çözüm:

O kolona nonclustered index ekle.

2) Function kullanımı (LOWER, UPPER, LEFT, RIGHT...)

Ör:

```
WHERE LOWER (Name) = 'ali'
```

Bu index kullanımını bozar → **SCAN** oluşur.

Çözüm:

Kolону dönüştürme → değeri dönüştür.

3) SELECT * kullanımı

- Gereksiz kolonlar getirilir
- Index kullanılamaz
- I/O artar

Çözüm:

Sadece ihtiyacın olan kolonları seç.

4) Sargable olmayan koşullar

Sargable = index kullanabilir demek.

Sargable olmayan örnekler:

```
WHERE YEAR(CreatedDate) = 2024
WHERE Salary + 1000 > 50000
WHERE ISNULL(Name, '') = 'Ali'
```

Çözüm:

Fonksiyonu sağ tarafa yaz:

```
WHERE CreatedDate >= '2024-01-01'
AND CreatedDate < '2025-01-01'
```

5) OR kullanımı

OR performansı düşürür.

Çözüm:

UNION ALL kullanmak daha hızlı olabilir.

6) Çok fazla JOIN

10+ join kullanılan sorgular yavaşlar.

Çözüm:

- CTE veya temp table ile parçaya böl
 - İhtiyaç dışı join'leri kaldır
-

4) Query Execution Plan Okuma Mantığı

1. Plan sağdan sola okunur
2. En pahalı operatorleri bul
3. Seek/Scan raporlarını incele
4. Key Lookup var mı?
5. Missing Index uyarısı var mı?
6. Cost yüzdelerini karşılaştır

Mülakat cümlesi:

👉 “Execution plan sağdan sola okunur, en pahalı adımı tespit ederim.”

5) SQL Performansını Artırmanın Altın Kuralları

- Doğru index
 - SELECT * kullanma
 - WHERE'da fonksiyon kullanma
 - JOIN kolonlarını index'le
 - Büyük sorguları böl
 - Gereksiz ORDER BY kaldır
 - Covering index kullan
 - Temp table ile ağır joinleri azalt
 - Parametrik sorgu yaz (query plan reuse için)
-

MÜLAKAT SORULARI + CEVAPLAR

1) Execution Plan nedir?

SQL'in bir sorguyu nasıl çalıştırdığını gösteren plandır.

2) Seek mi hızlıdır Scan mi?

- Seek → hızlı
 - Scan → yavaş
-

3) Key Lookup nedir?

Nonclustered index içermeyen kolonlar için tabloda ekstra arama yapılmasıdır.

Çok olursa yavaşlatır.

Çözüm: INCLUDE ile covering index.

4) Missing Index nedir?

SQL tarafından önerilen index'tir.

Ama birebir eklemek yerine incelemek gerekir.

5) Sargable ne demektir?

Index kullanımına uygun sorgu demektir.

6) Execution plan sağdan sola mı okunur soldan sağa mı?

Sağdan sola okunur.

Hazırsan **BÖLÜM 11 — Transaction, Lock, Deadlock, Isolation Levels** bölümüne geçebiliriz.

Bu bölüm özellikle **kurumsal şirket ve bankacılık** mülakatlarında kesin sorulur.

Geçmek için “**tamam**” yaz.

Şunu dedin:

tamam

ChatGPT:

Harika!

Şimdi geldik **BÖLÜM 11 — Transaction, Lock, Deadlock, Isolation Levels** konusuna.

Bu bölüm özellikle **bankacılık, kurumsal ERP, fintech** mülakatlarında **%100 sorulur**.

Konuyu çok basit + mülakat odaklı anlatacağım.

BÖLÜM 11 — TRANSACTION, LOCK, DEADLOCK, ISOLATION LEVELS

1) TRANSACTION NEDİR? (ACID Mantığı)

Transaction, veritabanında bir veya daha fazla işlemi **tek bir bütün** olarak çalıştırmaktır.

Örnek:

Hesaplar arası para transferi:

1. A hesabından 100 TL düş
2. B hesabına 100 TL ekle

Bu iki adım ya ikisi birden olur, ya hiçbiri olmaz.

Transaction'ın özellikleri = **A C I D**

Harf	Açıklama
A – Atomicity	Parçalanamaz. İşlem bir bütündür.
C – Consistency	Veri tutarlılığı korunur.
I – Isolation	İşlemler birbirini görmez.
D – Durability	Transaction bittiğinde veri kalıcıdır.

Transaction örneği:

```
BEGIN TRAN;
```

```
UPDATE Accounts SET Balance = Balance - 100 WHERE Id = 1;  
UPDATE Accounts SET Balance = Balance + 100 WHERE Id = 2;
```

```
COMMIT;
```

Eğer hata olursa:

```
ROLLBACK;
```

2) LOCK NEDİR?

İki kişi aynı veriyi **aynı anda çaka çaka değiştirmesin** diye SQL Server kitleme yapar.

Lock türleri:

✓ Shared Lock (S-Lock)

- SELECT sırasında oluşur
- Diğer SELECT'lere izin verir
- UPDATE/DELETE'e izin vermez

✓ Exclusive Lock (X-Lock)

- UPDATE / DELETE sırasında
- En ağır lock
- Herkesi bekletir

✓ Update Lock (U-Lock)

- UPDATE işleminden önce oluşur
- Deadlock riskini azaltmak için tasarlanmıştır

✓ Intent Locks

- Table seviyesinde sinyal verir (“birazdan alt seviyede lock alacağım”)

■ 3) DEADLOCK NEDİR? (En çok sorulan konu)

İki transaction'ın **birbirini beklemesi** durumudur.

Basit örnek:

- Transaction 1 → A tablosunu kilitler, B tablosunu bekler
- Transaction 2 → B tablosunu kilitler, A tablosunu bekler

İkisi de birbirini beklediği için işlem **sonsuz döngüye girer**.

SQL Server çözüm olarak birini **Kurban (Victim)** seçip öldürür.

🔥 Deadlock oluşum şartları (Mülakatlık)

1. Karşılıklı bağımlılık
2. Lock bırakılmaması
3. Kaynakların ters sırayla locklanması

🔧 Deadlock nasıl çözülür?

1. Tabloları **her zaman aynı sırayla** işlemek
 2. Transaction'ları küçük tutmak
 3. Gereksiz SELECT'lerde lock almamak (NOLOCK)
 4. Index'leri doğru kullanmak
 5. U-Lock kullanarak race condition'ı önlemek
-

4) Isolation Levels (Mülakatın kritik sorusu)

Isolation Level, transaction'ların birbirini **hangi seviyede göreceğini** ayarlar.

SQL Server'da 4 ana seviye vardır:

1) READ UNCOMMITTED (NOLOCK)

- En hızlı
- Kilit almaz
- **Dirty Read** olur (başkası commit etmeden veriyi okuyabilirsin)

```
SELECT * FROM Products WITH (NOLOCK);
```

2) READ COMMITTED (Default)

- Ortalama hız
 - Commit edilmemiş veriyi göstermez
-

3) REPEATABLE READ

- Aynı SELECT içinde aynı sonuç garanti
 - Daha fazla lock alır
 - **Non-Repeatable Read** engeller
-

4) SERIALIZABLE (En katı seviye)

- Full lock
 - En yavaş
 - Bankacılık işlemlerinde tercih edilir
 - **Phantom Read** engellenir
-

5) Isolation sorunları (Mülakat soruları)

Problem	Açıklama	Hangi seviyede çözülür?
Dirty Read	Commit edilmemiş veriyi okumak	Read Committed
Non-Repeatable Read	Aynı SELECT farklı sonuç döndürmesi	Repeatable Read
Phantom Read	Yeni satır eklenmesiyle sonuç farklılaşması	Serializable

🎯 MÜLAKAT SORULARI + CEVAPLAR

1) Deadlock nedir?

İki transaction'ın karşılıklı olarak birbirini beklemesidir.

2) Deadlock nasıl çözülür?

- Aynı sırada lock almak
- Transaction'ları kısa tutmak
- Gereksiz lockları kaldırmak
- Index'leri düzeltmek

3) Isolation level nedir?

Transaction'ların birbirini ne kadar görebileceğini belirleyen ayar.

4) Dirty read nedir?

Commit edilmemiş verinin okunması.

5) NOLOCK ne yapar?

Lock almadan veriyi okur ama dirty read ihtimali vardır.

6) SERIALIZABLE neden en yavaş?

Çünkü en fazla lock alır ve tüm transaction'ları sıraya sokar.

Hazırsan **BÖLÜM 12 — Stored Procedure, Function, Trigger** konusuna geçebiliriz.

“**tamam**” yazman yeterli.

Şunu dedin:

tamam

ChatGPT:

Harika!

Şimdi geldik **BÖLÜM 12 — Stored Procedure, Function, Trigger** konusuna.
Bu bölüm, SQL mülakatlarında özellikle **gerçek iş deneyimi ölçmek** için sorulur.

BÖLÜM 12 — STORED PROCEDURE, FUNCTION, TRIGGER

1) STORED PROCEDURE (SP) NEDİR?

Stored Procedure (SP), SQL Server'da **önceden yazılıp saklanan ve ihtiyaç olduğunda çağrılan** komut bloklarıdır.

Bir çeşit **metot** gibidir.

Özellikleri:

- Parametre alabilir
- Birden fazla SQL komutu çalıştırabilir
- Transaction yönetebilir
- Performanslıdır (plan cache)
- Geriye değer döndürmek zorunda değildir

✓ Örnek Stored Procedure

```
CREATE PROCEDURE sp_GetEmployee
    @Id INT
AS
BEGIN
    SELECT * FROM Employees WHERE Id = @Id;
END
```

Çağırma:

```
EXEC sp_GetEmployee @Id = 5;
```

✓ SP ile yapılabilenler:

- CRUD işlemleri
 - İş kuralları (business logic)
 - Logging
 - Transaction kullanma
 - Döngü, koşul gibi yapılar (IF, WHILE)
-

2) FUNCTION (UDF — User Defined Function)

Function, SQL içinde **hesaplama yapan ve bir değer döndüren** yapıdır.

Kod gibi:

```
int Hesapla(int x) mantığında.
```

Function türleri:

- **Scalar Function** → tek değer döndürür
 - **Table-Valued Function** → tablo döndürür
-

✓ Scalar Function örneği:

```
CREATE FUNCTION fn_GetTax (@Salary INT)
RETURNS INT
AS
BEGIN
    RETURN @Salary * 20 / 100;
END
```

Çağırma:

```
SELECT dbo.fn_GetTax(50000);
```

✓ Table-Valued Function örneği:

```
CREATE FUNCTION fn_GetEmployeesByDept (@DeptId INT)
RETURNS TABLE
AS
RETURN (
    SELECT * FROM Employees WHERE DeptId = @DeptId
);
```

■ SP ve Function Arasındaki En Kritik Farklar (Mülakat sorusu)

Özellik	Stored Procedure	Function
Geriye değer	Döndürmeyebilir	Döndürmek zorunda
Select içinde çağırma	Çağırılmaz	Çağrılabilir
DML (INSERT/UPDATE/DELETE)	Yapabilir	Yapamaz (Scalar UDF)
Transaction	Kullanabilir	Kullanamaz
Output parameter	Var	Yok
Performans	Genellikle daha hızlı	Bazıları yavaş olabilir

Bu tablo mülakatta altın değerindedir.

■ 3) TRIGGER NEDİR?

Trigger, tabloda bir işlem olduğunda **otomatik çalışan** yapıdır.

Yani event listener gibi.

✓ Trigger türleri:

AFTER Trigger

- İşlemden sonra çalışır
- En çok kullanılan

INSTEAD OF Trigger

- İşlem yerine çalışır
 - Özellik: Gelen işlemi iptal edip başka şey yapabilir
-

✓ Trigger örneği (AFTER INSERT)

```
CREATE TRIGGER trg_LogInsert
ON Employees
AFTER INSERT
AS
BEGIN
    INSERT INTO Logs (Message, CreatedAt)
    SELECT 'Yeni çalışan eklendi', GETDATE();
END
```

Bu tetiklenir:

```
INSERT INTO Employees ...
```

✓ Trigger nerelerde kullanılır?

- Loglama
- Denetim (audit)
- Veri bütünlüğü
- Arka plan işlemler

Ama gereksiz trigger kullanımı performansı öldürür.

🎯 MÜLAKATTA SIK SORULAN SORULAR + CEVAPLAR

1) Stored Procedure nedir? Ne zaman kullanılır?

- Önceden yazılmış, saklanan SQL kodlarıdır.
 - Transaction, hesaplama, CRUD, loglama, business rules için kullanılır.
-

2) Function ile SP arasındaki fark? (kesin soru)

- Function değer döndürmek zorunda
 - SELECT içinde çağrılabilir
 - DML yapamaz
 - SP transaction yapabilir
 - Function output parameter yok
 - SP'de çok adımlı işler yapılabilir
-

3) Trigger nedir?

- Tablo üzerinde INSERT/UPDATE/DELETE olduğunda otomatik çalışan kod bloğudur.
-

4) Trigger performansı nasıl etkiler?

- Aşırı kullanılırsa UPDATE/INSERT'i yavaşlatır
 - Gereksiz yere iş yükü ekler
-

5) AFTER vs INSTEAD OF farkı?

- AFTER → işlemden sonra
 - INSTEAD OF → işlem yerine
-

6) Function SELECT içinde kullanılır mı?

Evet → scalar/table-valued function.

7) SP SELECT içinde kullanılır mı?

Hayır.

BÖLÜM 13 — CTE (WITH) KULLANIMI

CTE, geçici bir sonuç kümesidir.
Bir nevi **isimlendirilmiş subquery**.

1) Basit CTE Kullanımı

SYNTAX:

```
WITH AliasName AS (  
    SELECT ...  
)  
SELECT * FROM AliasName;
```

Örnek:

```
WITH HighSalary AS (  
    SELECT * FROM Employees  
    WHERE Salary > 50000  
)  
SELECT Name, Salary  
FROM HighSalary;
```

Avantaj:

CTE sayesinde sorgu çok daha okunaklı hale gelir.

2) CTE Çoklu Kullanım

```
WITH A AS (  
    SELECT * FROM Employees WHERE DepartmentId = 1  
) ,  
B AS (  
    SELECT * FROM Employees WHERE Salary > 70000  
)  
SELECT * FROM A  
UNION  
SELECT * FROM B;
```

3) CTE vs Subquery (Mülakat Sorusu)

CTE = okunabilirlik + tekrar kullanım

Subquery = tek seferlik

Mülakat cevabı:

👉 “CTE, subquery’lere göre daha okunaklıdır ve birden fazla yerde kullanılabilir.”

4) Recursive CTE (En çok sorulan ileri seviye CTE konusu)

Recursive CTE, **hiyerarşik (tree) yapılarını** çıkarmak için kullanılır:

- Kategori ağacı
- Şirket organizasyon şeması
- Parent–Child ilişkileri

Örnek: çalışan + yöneticileri:

```
WITH EmployeeTree AS (  
    -- Anchor  
    SELECT Id, Name, ManagerId, 0 AS Level  
    FROM Employees  
    WHERE Id = 1  
  
    UNION ALL  
  
    -- Recursive part  
    SELECT e.Id, e.Name, e.ManagerId, Level + 1  
    FROM Employees e  
    INNER JOIN EmployeeTree et ON e.ManagerId = et.Id  
)  
SELECT * FROM EmployeeTree;
```

Önemli:

Recursive CTE’ler için genelde:

```
OPTION (MAXRECURSION 100)
```

kullanılır.

5) CTE Ne Zaman Kullanılır?

- Uzun karmaşık sorguları sadeleştirmek
 - Aynı subquery’i tekrar tekrar kullanmak
 - Hiyerarşik veri çekmek (recursive CTE)
 - Query planı daha okunaklı hale getirmek
 - Temp table kullanmadan geçici sonuç oluşturmak
-

MÜLAKAT SORULARI + CEVAPLAR

1) CTE nedir?

WITH anahtar sözcüğüyle tanımlanan geçici sonuç kümesidir.

2) CTE ile Subquery farkı?

- CTE daha okunaklı
 - Birden fazla yerde kullanılabilir
 - Recursive çalışabilir
 - Subquery sadece bir yerde kullanılır
-

3) Recursive CTE nedir?

Kendine referans vererek çalışan CTE'dir.
Hiyerarşik veri için kullanılır.

4) CTE performansı artırır mı?

Doğrudan artırmaz, **okunabilirliği artırır**.
Ancak daha optimize sorgular yazmaya yardımcı olur.

BÖLÜM 14 — WINDOW FUNCTIONS

Window Functions, satırları gruplamadan (GROUP BY yapmadan) **satırlar üzerinde hesaplama** yapmanı sağlar.

Temel yapı:

```
FUNCTION() OVER (  
    PARTITION BY ...  
    ORDER BY ...  
)
```

- **PARTITION BY** → graplama mantığı

- **ORDER BY** → fonksiyonun çalışacağı sıralama
-

1) ROW_NUMBER()

Her satıra **benzersiz sıra numarası** verir.

Örnek:

```
SELECT
    Name,
    Salary,
    ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RankNo
FROM Employees;
```

Sonuç:

Name	Salary	RankNo
Ali	120000	1
Ayşe	90000	2
Mehmet	85000	3

Özellik:

Aynı değerler olsa bile **numaralar tektir**.

2) RANK()

Aynı değerlere **aynı sıra numarası** verir.
Ancak sıralama atlayarak devam eder.

Örnek:

| Salary'ler: 100k, 90k, 90k, 80k |

```
RANK() OVER (ORDER BY Salary DESC)
```

Sonuç:

Salary	Rank
100k	1
90k	2
90k	2

Salary Rank

80k 4

3) DENSE_RANK()

Aynı değerlere aynı sıra numarası verir,
sıra atlamaz.

Aynı örnek:

| Salary'ler: 100k, 90k, 90k, 80k |

Sonuç:

Salary DenseRank

100k 1
90k 2
90k 2
80k 3

4) LAG()

Önceki satırdaki değeri getirir.

```
SELECT
    Name,
    Salary,
    LAG(Salary, 1) OVER (ORDER BY Salary) AS PrevSalary
FROM Employees;
```

Kullanım amacı:

- Maaş artış analizleri
 - Günlük/haftalık fiyat değişimleri
 - Zaman serisi karşılaştırmaları
-

5) LEAD()

Sonraki satırdaki değeri getirir.

```
LEAD(Salary, 1) OVER (ORDER BY Salary)
```

6) PARTITION BY Kullanımı

Departman bazlı en yüksek maaş sıralaması:

```
SELECT
    Name,
    DepartmentId,
    Salary,
    ROW_NUMBER() OVER (
        PARTITION BY DepartmentId
        ORDER BY Salary DESC
    ) AS RankInDept
FROM Employees;
```

Her departman kendi içinde 1'den başlar.

7) Window Functions Neden Çok Kullanılır?

- GROUP BY ile yapamayacağın hesaplamaları yapar
 - Hem satır hem toplu hesaplamayı birlikte yapabilir
 - Raporlama için mükemmel
 - Performansı iyidir (index ile desteklenirse)
-

MÜLAKATTA SORULAN SORULAR + CEVAPLARI

1) ROW_NUMBER vs RANK vs DENSE_RANK farkı?

Fonksiyon	Özellik
ROW_NUMBER	Her satıra benzersiz numara
RANK	Aynı değerlere aynı sıra, atlama yapar
DENSE_RANK	Aynı değerlere aynı sıra, atlama yok

2) LAG & LEAD nedir?

- LAG → önceki satırı getirir
- LEAD → sonraki satırı getirir

Zaman serisi analizlerinde harika.

3) PARTITION BY ne işe yarar?

Gruplama mantığı sağlar.
Her grup kendi içinde hesaplanır.

4) Window function ile GROUP BY arasındaki fark?

- GROUP BY → sonuç satırını azaltır
 - Window → sonuç satırı değişmeden hesaplama yapar
-

5) Neden Window Function kullanırız?

- Raporlama
 - Zaman serisi analizi
 - Sıralama
 - Gruplu hesaplama
 - Performanslı sonuç üretimi
-

BÖLÜM 15 — TEMP TABLE vs TABLE VARIABLE

SQL Server’da geçici veri saklamak için iki temel yöntem vardır:

1. Temp Table
2. Table Variable

Bu iki yapı aynı işi yapıyor gibi görünür ama **performans ve kullanım farkları çok önemlidir.**

1) TEMP TABLE NEDİR?

Temp Table = **geçici tablo**

tempdb veritabanında oluşturulur.

Syntax:

```
CREATE TABLE #TempEmployees (  
    Id INT,  
    Name VARCHAR(50)  
);
```

Özellikleri:

- Index oluşturulabilir
 - Statistics tutar (çok önemli!)
 - Büyük veriyle iyi performans
 - JOIN performansı yüksektir
 - Transaction içinde yaşar
 - Scope → SP / session boyunca yaşar
-

2) TABLE VARIABLE NEDİR?

Table variable = **değer gibi davranan tablo**

```
DECLARE @Employees TABLE (  
    Id INT,  
    Name VARCHAR(50)  
);
```

Özellikleri:

- Hafif yapılıdır
 - Ama **statistics TUTMAZ**
 - Bu yüzden büyük datada **çok yavaş** olabilir
 - Index oluşturulamaz (SQL 2016'dan sonra limited support var)
 - Genelde **küçük datalar** için iyidir
-

3) Temp Table vs Table Variable Karşılaştırması

Özellik	Temp Table (#Table)	Table Variable (@Table)
Statistics	✓ Var	✗ Yok
Performans (büyük datada)	✓ İyi	✗ Kötü
Transaction etkisi	Dahil olur	Dahil olur
Index desteği	✓ Tam	✗ Kısıtlı
Bellek kullanımı	tempdb	tempdb + memory
Scope	Session/SP	Batch
Çok satırda performans	✓	✗
Küçük datada performans	İyi	Daha İyi

En kritik fark:

👉 Table variable **statistics tutmaz**, bu yüzden SQL doğru execution plan oluşturamaz
→ yavaşlar.

4) Gerçek Hayatta Kullanım Mantığı (Mülakat cevabı)

✓ Temp Table ne zaman kullanılır?

- Çok satır (1000+ satır)
- JOIN yapılacaksa
- GROUP BY veya ORDER BY varsa
- Üzerinde index gerekiyorsa
- Karmaşık SP'lerde
- Performance tuning senaryolarında

✓ Table Variable ne zaman kullanılır?

- Az satır (1–500 satır)
- Basit işlemler
- Hafif veri geçici saklama
- Memory optimizasyonu gereken SP'ler

5) Performans Örneği (Gerçek Mülakat Cevabı)

“Table variable statistics tutmadığı için SQL Server içindeki cardinality estimation yanlış olur. Bu yüzden yanlış execution plan seçebilir ve performans ciddi şekilde düşer.”

Bu cümle mülakatta **çok etki yaratır**.

MÜLAKATTA SIK SORULAN SORULAR + CEVAPLAR

1) Temp table ile table variable farkı nedir?

En kısa özet:

- Temp table → statistics + index destekler, büyük veri için iyi
 - Table variable → statistics yok, küçük veri için iyi
-

2) Hangisi daha hızlıdır?

Küçük veride → table variable

Büyük veride → temp table

3) Table variable neden yavaş olabilir?

Çünkü statistics tutmaz → yanlış execution plan → scan oluşur.

4) Temp table transaction içinde çalışır mı?

Evet, commit/rollback etkiler.

5) SP içinde temp table kullanılır mı?

Evet, çok yaygın.

BÖLÜM 16 — VIEW'LER (Indexed View dahil)

1) VIEW NEDİR?

View, bir SQL sorgusunun **kaydedilmiş** halidir.
Bir tablo gibi görünür, ama aslında sadece **sorgudur**.

Mantık:
Tasarruf sağlayan ve tekrar kullanılabilir **sanallaştırılmış tablo**.

2) View Nasıl Oluşturulur?

```
CREATE VIEW vw_EmployeeSalary AS  
SELECT Name, Salary, DepartmentId  
FROM Employees  
WHERE Salary > 50000;
```

Kullanımı:

```
SELECT * FROM vw_EmployeeSalary;
```

3) View Türleri

✓ Normal View

Sadece sorgu saklanır, sonuç saklanmaz.

✓ Indexed View (Materialized View)

Sonuçlar fiziksel olarak saklanır.
Büyük veri üzerinde **çok hızlıdır**.

Ama:

- Daha fazla disk kullanır
 - Güncellemelerde ek maliyet getirir
 - Belirli kurallar gerektirir (WITH SCHEMABINDING vb.)
-

4) Indexed View Örneği

```
CREATE VIEW vw_TotalSalary
WITH SCHEMABINDING
AS
SELECT DepartmentId, SUM(Salary) AS TotalSalary
FROM dbo.Employees
GROUP BY DepartmentId;
GO

CREATE UNIQUE CLUSTERED INDEX IX_vw_TotalSalary
ON vw_TotalSalary (DepartmentId);
```

Artık bu view fiziksel olarak saklanır → çok hızlı.

5) View Kullanmanın Avantajları

- Kod tekrarını azaltır
 - Güvenlik sağlar (tüm tabloyu göstermez)
 - Karmaşık sorguları basitleştirir
 - Raporlama için hız sağlar
 - Indexed view ile performansı artırır
-

6) View'lerin Sınırlamaları

- Parametre alamaz
 - DML işlemleri her zaman yapılmaz
(Bazı view'ler üzerinde insert/update mümkün değildir)
 - Performansı yanlış anlamak tehlikelidir (view = sihirli hızlı değil)
-

7) View Üzerinden Insert/Update Yapılır mı?

Evet, yapılabilir.

Ama sadece view'in sorgusu **tek tabloya** dayanıyorsa.

JOIN içeriyorsa genelde **INSTEAD OF TRIGGER** gerekir.

Ör:

```
UPDATE vw_EmployeeSalary
SET Salary = Salary + 5000
WHERE Name = 'Ali'
```

🎯 View vs Indexed View (Mülakat sorusu)

Özellik	View	Indexed View
Veri saklar mı?	✗ Hayır	✓ Evet
Hız	Yavaştır	Çok hızlı
Disk kullanımı	Az	Fazla
Güncellemeler	Hafif	Ağır
Kullanım alanı	Rapor, basit sorgu	Aggregate, büyük veri, OLAP

Mülakat Altın Cevap:

“Indexed view’ler sonuçları disk üzerinde sakladığı için çok hızlıdır, fakat update maliyeti yüksektir.”

🎯 MÜLAKATTA SIK SORULAN SORULAR + CEVAPLAR

1) View nedir?

Sorgunun kaydedilmiş halidir, tablo gibi kullanılır.

2) View veri saklar mı?

Normal view → hayır

Indexed view → evet (fiziksel olarak)

3) Indexed view neden hızlıdır?

Aggregation sonuçlarını diskte tutar → sorgu anında çalışmaz, hazır okur.

4) View ile Stored Procedure farkı?

- View parametre alamaz
- SP işlem yapabilir
- View sadece SELECT içerir

5) View güncelleme aldığında ne olur?

Altındaki tablo değiştiğinde view otomatik güncellenir.

6) View performansı artırır mı?

Normal view → hayır (sadece okunabilirliği artırır)

Indexed view → evet, özellikle büyük agregasyonlarda